ASIF SAAS STUDIO

HD Image Generator

Complete documentation: user guide, processing pipeline, validation rules, architecture, deployment and testing for the browser-only HD image enhancement application.

Version: 1.0 · **Category:** Design
Created by: Dr. Asif Ahmad
Date: 20 October 2026
Deliverable: a single self-contained `index.html` (22.4 KB) — no server, no build step, no accounts, no uploads.

1 · Introduction

HD Image Generator turns ordinary photos into crisp, high-definition images — entirely inside the visitor's browser. The user supplies a photo; the app enlarges it with high-quality resampling and applies denoise, colour correction and sharpening; the finished image is returned as a download. Nothing is uploaded anywhere.

The application was designed for the Asif SaaS Studio tools catalogue, where every tool runs inside a sandboxed iframe on a Wix Headless site. That environment forbids API keys, server calls the user hasn't approved, browser storage, service workers and permission prompts — so HD Image Generator was built as one self-contained HTML file: all CSS and JavaScript inline, zero external assets, zero network requests, and a strict no-storage, no-permissions design.

> **Honest scope.** This is *smart enhancement*, not AI super-resolution. High-quality upscaling plus sharpening, denoise and colour correction make reasonably sharp photos visibly crisper — but no browser-only tool can invent fine detail that was never captured. True AI upscaling requires a cloud AI service with private API keys and a backend; Section 11 explains that option and its deployment plan honestly instead of pretending the HTML file implements it.

Who it is for

- Students and teachers preparing images for presentations, assignments and print.
- Small businesses enlarging product or catalogue photos for a website.
- Anyone who wants a sharper, larger image without uploading private photos to a stranger's server.

2 · Features at a glance

Feature	What it does
Upscale 1× / 2× / 3× / 4×	Progressive high-quality resampling (in 1.5× steps) to the chosen factor — 2× is labelled “HD”, 4× “Ultra”.
Sharpen slider (0–100)	3×3 convolution sharpening whose strength follows the slider; edge-aware, alpha preserved.
Gentle denoise	Optional 35% blend with a 3×3 gaussian blur, applied *before* sharpening so grain is not amplified.
Colour correction	Brightness, contrast and saturation sliders with photographic defaults (contrast 100, saturation 105).
Three output formats	PNG (lossless), JPEG (adjustable quality 60–100, default 92), WebP.
Drag & drop + browse	Drop zone with keyboard access, plus a standard file picker.
Full input validation	Type allow-list, 25 MB limit, empty/corrupt file detection — each failure explained in plain language (Section 5).
Memory safety caps	Output capped at 8000 px per side and 40 megapixels; a visible note appears whenever a cap applies.
Privacy by design	The image never leaves the device. No uploads, analytics, storage or tracking of any kind.
Responsive layout	Two-column studio layout that stacks cleanly on phones (~380 px) used inside the Wix iframe.

3 · Quick start

1. Open `index.html` in any modern browser — or open the tool on the Asif SaaS Studio site. No installation.
2. Drop a photo onto the drop zone, or tap it and pick a file.
3. Check the preview and file details that appear under the drop zone.
4. Keep the defaults (2× HD, sharpen 45, denoise on, JPEG 92) or adjust the sliders.
5. Tap **Generate HD Image**. A few seconds later the result preview, final dimensions and file size appear.
6. Tap **Download HD Image**. The file saves as `<original-name>-HD.jpg` (extension follows the chosen format).

> **Tip.** For most photos the defaults are the sweet spot. Push Sharpen above ~60 only for genuinely soft images — over-sharpening adds halos around edges.

4 · User guide — the controls in detail

4.1 Choosing an image

The drop zone accepts PNG, JPEG, WebP, BMP and GIF files up to 25 MB. GIFs are processed as their first frame. When a file is accepted, the app shows a thumbnail preview with the filename, the pixel dimensions and the file size — always verify this is the image you intended before generating.

4.2 Upscale factors

Choice	Meaning	Best for
1× Enhance	No size change; enhancement only	Already-large photos that just look dull or soft
2× HD	Doubles each dimension (4× the pixels)	The default — web, slides and social media
3×	Triples each dimension	Small images headed for full-screen display
4× Ultra	Quadruples each dimension	Print from small originals; expect the safety caps to engage sooner

4.3 Enhancement sliders

Slider	Range	Default	Effect
Sharpen	0–100	45	Strength of the sharpening convolution (mapped to kernel amount × 0.9).
Contrast	50–150	100	100 is neutral; higher deepens shadows and brightens highlights.
Colour saturation	0–200	105	0 gives true greyscale; 200 doubles colour intensity.
Brightness	50–150	100	100 is neutral; shifts every pixel by up to ±127 levels.
Gentle denoise	On/Off	On	Smooths grain before sharpening. Turn off for screenshots and line art.

4.4 Output

PNG is lossless and best for graphics, screenshots and further editing — but produces the largest files. **JPEG** is the right default for photographs; the Quality slider (60–100, default 92) trades size against fidelity. **WebP** gives the smallest files and is ideal for web use. The result panel always reports the final dimensions, format and exact file size before you download.

5 · Input validation

Every input is checked before any processing happens, and every rejection explains itself in plain language next to the drop zone. The same rules are enforced by automated tests (Section 9).

Check	Rule and message
File present	Generating with no image asks the user to choose one first; the Generate button stays disabled until a valid image loads.
File type	Allow-list: PNG, JPEG, WebP, BMP, GIF. Anything else: “That file is not a supported image…”
SVG	Rejected with its own explanation — vector files need different handling than photos.
Empty file	0-byte files: “That file appears to be empty.”
File size	Over 25 MB: rejected before loading, protecting the device's memory.
Corrupt / unreadable	If the browser cannot decode the image (or reports 0 dimensions): “That image could not be read. It may be corrupt…”
Output size	Requested outputs above 8000 px per side or 40 MP are reduced to the cap and the note is shown with the final size — never a silent change.
Processing failure	Any unexpected failure (typically extreme sizes on low-memory phones) produces a plain error advising a lower upscale factor.

6 · How it works — the processing pipeline

All processing uses the browser's built-in Canvas 2D and typed-array pixel maths. No libraries, no workers, no uploads. The pipeline runs in five stages:

1. **Decode & measure.** The file is decoded by the browser's image engine (which also honours EXIF orientation in modern browsers) and its true pixel size is recorded.

2. **Progressive upscale.** Instead of one large jump, the image is drawn upward in ≤1.5× steps with `imageSmoothingQuality = "high"`. Stepwise resampling preserves edges noticeably better than a single 4× stretch.

3. **Denoise (optional).** A 3×3 gaussian blur (kernel 1-2-1 / 2-4-2 / 1-2-1, divisor 16) is blended at 35% into the image — enough to calm sensor grain without softening real detail, and deliberately done *before* sharpening.

4. **Colour correction.** One per-pixel pass applies saturation (luma-weighted), then contrast, then brightness (formulas below).

5. **Sharpen & export.** A 3×3 sharpening convolution emphasises edges; the canvas is then encoded to the chosen format/quality and offered as a download.

6.1 The mathematics

Saturation — each pixel is moved toward or away from its own luminance:

$$L = 0.2126 \cdot R + 0.7152 \cdot G + 0.0722 \cdot B \quad \cdot \quad C' = L + (C - L) \cdot s$$

Contrast — the classic photographic factor, equal to exactly 1 at the neutral setting:

$$F = \frac{259 \cdot (L_c + 255)}{255 \cdot (259 - L_c)}, \qquad L_c = (\text{pct} - 100) \cdot 1.28$$

Brightness — a uniform shift, $\pm(\text{pct} - 100) \cdot 2.55$ levels, applied after contrast. All channel values are clamped to 0–255 after every stage.

Sharpen kernel — for slider amount *a* (0–1, scaled ×0.9):

$$\begin{bmatrix} 0 & -a & 0 \\ -a & 1+4a & -a \\ 0 & -a & 0 \end{bmatrix}$$

The kernel sums to exactly 1, so flat areas pass through unchanged (verified by test 6) and only genuine edges are emphasised. Convolution reads from a copy of the pixels — never from partially-written output — and the image border is passed through untouched rather than wrapped or mirrored.

6.2 Safety caps

Cap	Value	Why

Maximum side	8000 px	Canvas implementations become unreliable beyond this on common devices.
Maximum pixels	40 MP	Each processing stage holds full RGBA buffers (≈160 MB at 40 MP); the cap keeps phones responsive.
Maximum input file	25 MB	Decode memory grows far beyond file size; the limit rejects risky inputs early.

7 · Architecture & sandbox compliance

The deliverable is a single HTML file (22,398 bytes) in three layers: a presentation layer (inline CSS, responsive grid, brand palette — emerald `#103E32`, lime `#DCF5B5`, paper `#F7F8F2`), a user-interface layer (drop zone, controls, previews, messages) and a **pure image-maths core** — four functions with no DOM access (`adjustPixels`, `convolve3x3`, `sharpenKernel`, `effectiveSize`, `validateFile`) that are unit-tested directly (Section 9).

Sandbox rule	How the app complies
No API keys or private data	None exist anywhere in the file; there is nothing to leak.
No localStorage / cookies	Never used. All state lives in ordinary JavaScript variables for the session only.
No service workers	None registered; the file works from `file://` or any static host.
No browser permissions	No camera, microphone, notification, geolocation or clipboard APIs are touched. File access happens only through the user's own picker/drop.
No parent-window access	The script never references `window.parent` or `postMessage`; it behaves identically standalone or embedded.
No network calls	Zero external assets and zero fetches; downloads use in-memory object URLs created from the processed canvas.

Temporary object URLs are revoked when replaced or reset, so repeated use does not leak memory.

8 · Deployment

8.1 Asif SaaS Studio (Wix Headless + CMS)

1. Wix dashboard → CMS → **SaasProducts** → add a new item.
2. Title: *HD Image Generator — Upscale & Enhance Photos*; Slug: `hd-image-generator`; Category: *Design*; Version: *1.0*; Demo: Yes; Published: Yes.
3. Paste the complete contents of `index.html` into the **appHtml** field.
4. Open `app.html?slug=hd-image-generator` on the live site and run the smoke test in Section 9.3.

8.2 Any other host

Upload `index.html` to any static host (GitHub Pages, Netlify, a school server) or open it directly from disk. There is no build step and no configuration.

8.3 GitHub

The source, this documentation and the test report are versioned in the `hd-image-generator` repository under the owner's account — the repository README carries the short description, feature list and test summary.

9 · Testing

9.1 Automated tests — 18/18 passed

The pure core functions were extracted from the shipped file itself and executed in Node.js against synthetic pixel data — so the tests certify the exact code users run:

#	Test	Result
1	Neutral settings (100/100/100) leave every pixel unchanged	Pass
2	Brightness +50 raises pixel values	Pass
3	Saturation 0 produces true greyscale (R=G=B)	Pass
4–5	Contrast 150 darkens dark tones; contrast 50 lifts them	Pass
6	Identity convolution returns the image byte-for-byte	Pass
7	Sharpening a flat image changes nothing (no invented artefacts)	Pass
8	Gaussian kernel on a flat image changes nothing	Pass
9	Sharpening widens a real edge (dark side 50→0, bright side 200→255)	Pass
10	2× sizing is exact (1000×800 → 2000×1600, uncapped)	Pass
11	Pixel cap engages (6000×6000 @4× → 6324×6324 = 40 MP)	Pass
12	Side cap engages (5000×1000 @4× → 8000×1600)	Pass
13–18	Validation: missing file, valid PNG, SVG rejection, non-image rejection, 26 MB rejection, empty-file rejection	Pass

The complete inline script also passes a JavaScript syntax check, and the file size was verified at build time (22.4 KB against the 150 KB ceiling).

9.2 Manual checklist

- Open `index.html` in Chrome/Edge — loads cleanly, no console errors.
- Upload a small JPEG → preview, filename, dimensions and size appear.
- Drop a `.txt` file → clear “not a supported image” error.
- Try an SVG → dedicated “SVG files are not supported” message.
- Generate at 2× with defaults → result preview shows doubled dimensions.
- Compare Sharpen 0 vs 100 on a soft photo → visible crispness difference.
- Download PNG, JPEG and WebP → files save as `<name>-HD.<ext>`.
- Very large photo at 4× → capped-size note appears; app stays responsive.
- Narrow window (~380 px) → layout stacks; every control reachable.

9.3 Live smoke test (after CMS publishing)

Open the live app page, process one known photo at 2×, and confirm: the app loads inside the iframe, the download saves, and the result dimensions are exactly double the original.

10 · Troubleshooting & FAQ

Question	Answer
“File is not a supported image”	The file is not PNG/JPEG/WebP/BMP/GIF — or its type was misreported by the device. Convert it to PNG or JPEG and retry.
The result doesn't look bigger	Check the result panel: it prints the exact output dimensions. If a cap engaged, the note explains the final size.
Edges look harsh / haloed	Sharpen is too high for that photo. Drop it to 20–35 and regenerate.
The photo looks softer, not sharper	Denoise may be fighting a genuinely grainy original, or the source is very low-resolution — enhancement cannot recreate missing detail (Section 1).
Processing is slow	Large images at 4× move hundreds of megabytes through memory. Use 2×, or a device with more RAM.
My photo opened sideways	Modern browsers honour EXIF orientation automatically. If an older browser doesn't, rotate the photo in any editor first.
Is my photo uploaded anywhere?	No. Processing is 100% on-device; the page makes no network requests at all.
The download didn't start in the embedded page	Open the tool in a full browser tab — some embedded contexts restrict downloads.

11 · Limitations & the optional AI upgrade

11.1 Known limitations of v1.0

- Enhancement is mathematical, not generative: faces, text and fine textures improve modestly, not magically.
- Animated GIFs are processed as a single (first) frame.
- Very large outputs are capped (8000 px / 40 MP) for device safety.
- No batch mode — one image at a time, by design, to keep memory use predictable.
- Colour work is global; there are no local adjustments (no selective brightening of faces, for example).

11.2 If true AI upscaling is ever wanted — deployment plan

A future “AI HD” tier would need: (1) a small backend service (e.g., a Node/Python app on a VPS or a serverless function) holding the AI provider's key server-side; (2) an upload endpoint that forwards the image to the AI upscaler and returns the result; (3) rate-limiting and a file-size policy, because AI upscaling is billed per image; (4) a privacy notice, because photos would leave the user's device — a real change from v1.0's promise. The single-file app would then call that endpoint instead of the local pipeline. Until such a backend exists, v1.0 deliberately makes no such claim.

12 · Version history & credits

Version	Date	Notes
1.0	20 Oct 2026	First release: progressive upscaling 1×–4×, denoise, colour correction, sharpening, three output formats, full validation, safety caps. 18/18 automated tests passed.

Designed and built for **Asif SaaS Studio**. Created by **Dr. Asif Ahmad**. The application contains no third-party code, libraries or assets.

Appendix A · The pure image-maths core (verbatim)

These are the exact functions tested in Section 9, reproduced from the shipped `index.html`. The complete file — interface markup, styles and wiring included — ships as `index.html` in the project repository.

```
function clamp255(v){ return v < 0 ? 0 : (v > 255 ? 255 : v); }

// Per-pixel brightness / contrast / saturation. Values are percentages (100 = neutral).
function adjustPixels(data, brightnessPct, contrastPct, saturationPct){
  var b = (brightnessPct - 100) * 2.55;                 // -127.5 .. +127.5
  var L = (contrastPct - 100) * 1.28;                   // -64 .. +64 over the 50..150 slider range
  var contrastF = (259 * (L + 255)) / (255 * (259 - L)); // = 1 exactly at 100
  var s = saturationPct / 100;
  for (var i = 0; i < data.length; i += 4){
    var r = data[i], g = data[i + 1], bl = data[i + 2];
    // saturation (luma-based)
    var lum = 0.2126 * r + 0.7152 * g + 0.0722 * bl;
    r = lum + (r - lum) * s;
    g = lum + (g - lum) * s;
    bl = lum + (bl - lum) * s;
    // contrast then brightness
    r = clamp255((r - 128) * contrastF + 128 + b);
    g = clamp255((g - 128) * contrastF + 128 + b);
    bl = clamp255((bl - 128) * contrastF + 128 + b);
    data[i] = r; data[i + 1] = g; data[i + 2] = bl;
  }
  return data;
}

// 3x3 convolution; reads from a copy so edges don't feed back. Alpha untouched.
function convolve3x3(src, dst, w, h, kernel, divisor){
  for (var y = 0; y < h; y++){
    for (var x = 0; x < w; x++){
      var idx = (y * w + x) * 4;
      if (x === 0 || y === 0 || x === w - 1 || y === h - 1){
        dst[idx] = src[idx]; dst[idx + 1] = src[idx + 1]; dst[idx + 2] = src[idx + 2]; dst[idx + 3] = src[idx
+ 3];
        continue;
      }
      var accR = 0, accG = 0, accB = 0, k = 0;
      for (var ky = -1; ky <= 1; ky++){
        for (var kx = -1; kx <= 1; kx++){
          var p = ((y + ky) * w + (x + kx)) * 4, kv = kernel[k++];
          accR += src[p] * kv; accG += src[p + 1] * kv; accB += src[p + 2] * kv;
        }
      }
      dst[idx]     = clamp255(accR / divisor);
      dst[idx + 1] = clamp255(accG / divisor);
      dst[idx + 2] = clamp255(accB / divisor);
      dst[idx + 3] = src[idx + 3];
    }
  }
  return dst;
}

// Sharpen kernel for amount a (0..1): centre 1+4a, neighbours -a.
function sharpenKernel(a){
  return [0, -a, 0,  -a, 1 + 4 * a, -a,  0, -a, 0];
}
var GAUSS_KERNEL = [1, 2, 1,  2, 4, 2,  1, 2, 1]; // divisor 16

// Effective output size for a source and requested scale, honouring caps.
function effectiveSize(srcW, srcH, scale, maxSide, maxPixels){
  var tw = Math.max(1, Math.round(srcW * scale));
  var th = Math.max(1, Math.round(srcH * scale));
  var capped = false;
  if (tw * th > maxPixels){
    var f = Math.sqrt(maxPixels / (tw * th));
    tw = Math.max(1, Math.floor(tw * f)); th = Math.max(1, Math.floor(th * f)); capped = true;
  }
  if (tw > maxSide){ th = Math.max(1, Math.floor(th * maxSide / tw)); tw = maxSide; capped = true; }
  if (th > maxSide){ tw = Math.max(1, Math.floor(tw * maxSide / th)); th = maxSide; capped = true; }
  return { w: tw, h: th, capped: capped };
```

```
}

// File validation. Returns "" when valid, else a human-readable error.
function validateFile(file){
  if (!file) return "Please choose an image file first.";
  var okTypes = ["image/png", "image/jpeg", "image/webp", "image/bmp", "image/gif"];
  if (file.type === "image/svg+xml") return "SVG files are not supported — please use a PNG, JPEG, WebP, BMP
or GIF image.";
  if (okTypes.indexOf(file.type) === -1) return "That file is not a supported image. Please use PNG, JPEG,
WebP, BMP or GIF.";
  if (file.size === 0) return "That file appears to be empty. Please choose another image.";
  if (file.size > 25 * 1024 * 1024) return "That image is larger than 25 MB. Please choose a smaller file.";
  return "";
}
```